Elasticsearch: Mapping & Analysis

- Overview of how data is structured and processed for search

What is Mapping?

- **First: What is Mapping?**
- ✅ **Definition**
- **Mapping = the structure (schema) of your index**
- It tells Elasticsearch:
- What fields exist
- What type each field has
- How each field should be indexed and stored
- 👉Think of it like a **table schema in a database**

Why Mapping is Important?

- Affects search accuracy
- Controls sorting and aggregation
- Prevents wrong automatic type detection

Example of dynamic Mapping

```
</> JSON

POST /books/_doc
{
  "title": "Machine Learning Basics",
  "price": 100,
  "published": "2024-01-01"
}
```

- Elasticsearch will **automatically guess types**:
- title → text
- price → float
- published → date
- ⚠ This is called **dynamic mapping** (not always safe)

Example With Explicit Mapping

```
PUT /books
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text"
      },
      "author": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      },
      "published": {
        "type": "date"
      }
    }
  }
}
```

Text vs Keyword

- text: analyzed, used for search
- keyword: exact match, used for filtering & sorting

🔥 Key Difference

Type	Behavior
text	analyzed (split into words)
keyword	exact value (no splitting)

Mapping Example

```
PUT /products
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      },
      "created_at": {
        "type": "date"
      }
    }
  }
}
```

What is Analysis?

- ✓ **Definition**
- **Analysis = the process of converting text into searchable tokens**
- It happens:
- When indexing data
- When searching
- We use it Because computers don’t understand sentences — they understand tokens (words)

Analysis Pipeline

- 1. Tokenizer → splits text
- 2. Filters → modify tokens
- 3. Analyzer = tokenizer + filters

Example

1. Tokenizer

Splits text into words

Example:

```
"AI is cool"
↓
["AI", "is", "cool"]
```

2. Token Filters

Modify tokens

Examples:

- lowercase → "AI" → "ai"
- remove stop words → "is" removed

Result:

```
["ai", "cool"]
```

3. Analyzer = Tokenizer + Filters

Analyzer

- Allows control over tokenization
- Can remove stopwords
- Can apply lowercase, stemming, etc.
- ◇ **Built-in Analyzer Example**
- Default analyzer (standard):
- splits text
- converts to lowercase
- removes punctuation

Custom analyzer

```
PUT /articles
{
  "settings": {
    "analysis": {
      "analyzer": {
        "my_analyzer": {
          "type": "standard",
          "stopwords": "_english_"
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "content": {
        "type": "text",
        "analyzer": "my_analyzer"
      }
    }
  }
}
```

Mapping vs Analysis

- Mapping: structure & types
- Analysis: text processing
- Together → control search behavior

🔥 Mapping vs Analysis (Core Idea)

Concept	Meaning
Mapping	defines structure + types
Analysis	defines how text is processed

Final Summary

- Mapping controls **how data is stored**
- Analysis controls **how text is understood**
- Both together control **search quality**

Full Example

- **Step 1: Create Index (Mapping + Analysis)**

```
PUT /articles
{
  "settings": {
    "analysis": {
      "analyzer": {
        "my_analyzer": {
          "type": "standard",
          "stopwords": "_english_"
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "my_analyzer"
      },
      "category": {
        "type": "keyword"
      }
    }
  }
}
```

Step 2:Insert Data

- POST /articles/_doc/1
- {
- "title": "Machine Learning is Amazing",
- "category": "AI"
- }

- POST /articles/_doc/2
- {
- "title": "Deep Learning and AI",
- "category": "AI"
- }

- POST /articles/_doc/3
- {
- "title": "Cooking Recipes for Beginners",
- "category": "Food"
- }

Step 3:TEST ANALYSIS

- GET /articles/_analyze
- {
- "analyzer": "my_analyzer",
- "text": "Machine Learning is Amazing"
- } ✅ Expected Output:

```
JSON
["machine", "learning", "amazing"]
```

Test search

- GET /articles/_search
- {
- "query": {
- "match": {
- "title": "learning"
- }
- }
- }

✅ **Result:**

- Document 1 ✔
- Document 2 ✔

👉 Because both contain token **learning**

Test partial search

- GET /articles/_search
- {
- "query": {
- "match": {
- "title": "machine"
- }
- }
- }
- ✓ Matches doc 1 only